

Manipulation des données réelles avec Python (Pandas)

Statistique Appliquée/ Djibril NDIAYE (UCAD)

1 Introduction

Dans un contexte marqué par la disponibilité croissante de données dans des domaines variés tels que l'économie, la finance, la santé ou encore l'ingénierie, la capacité à manipuler, analyser et interpréter des données constitue une compétence fondamentale.

Les données réelles sont généralement organisées sous forme de tableaux, souvent volumineux, hétérogènes et parfois incomplets. Leur exploitation nécessite l'utilisation d'outils adaptés permettant de structurer l'information, de corriger les imperfections et d'en extraire des connaissances pertinentes.

Dans ce chapitre, nous nous intéressons à la manipulation des données à l'aide de la bibliothèque **Pandas** du langage Python. Cette bibliothèque constitue aujourd'hui une référence en analyse de données, en raison de sa simplicité d'utilisation et de la richesse de ses fonctionnalités.

L'objectif est de transformer des données brutes en informations exploitables dans un cadre statistique, économique ou décisionnel. Cette transformation repose sur une démarche méthodologique rigoureuse, articulée autour de plusieurs étapes fondamentales.

Démarche générale

L'analyse d'un jeu de données suit généralement les étapes suivantes :

- **Exploration initiale des données** : compréhension de la structure du tableau, identification des variables et détection des anomalies ;
- **Nettoyage des données** : traitement des valeurs manquantes, correction des erreurs et suppression des incohérences ;
- **Transformation des données** : création de nouvelles variables et adaptation des données à l'objectif de l'étude ;
- **Analyse descriptive** : utilisation d'indicateurs statistiques pour résumer et caractériser les données ;
- **Analyse des relations** : étude des dépendances entre variables (corrélation, covariance, etc.) ;
- **Visualisation** : représentation graphique des données pour faciliter leur interprétation.

Objectifs du chapitre

Ce chapitre vise à :

- maîtriser les outils de manipulation de données avec Pandas ;
- comprendre les enjeux liés à la qualité des données ;
- développer une approche rigoureuse de l'analyse statistique ;
- établir un lien entre outils informatiques et interprétation statistique.

Positionnement du chapitre

Les notions développées dans ce chapitre constituent un préalable indispensable aux méthodes statistiques plus avancées, notamment en statistique inférentielle, en économétrie et en science des données.

Une bonne maîtrise de ces outils garantit la fiabilité des analyses ultérieures et la pertinence des conclusions tirées à partir des données.

Un jeu de données de référence sera utilisé tout au long du chapitre afin d'illustrer les différentes méthodes.

2 Présentation du jeu de données

Afin d'illustrer les différentes manipulations présentées dans ce chapitre, nous utiliserons un jeu de données simple représentant des informations sur des individus.

Ce tableau contient les variables suivantes :

- **age** : âge de l'individu ;
- **note** : note obtenue à un examen ;
- **sexe** : sexe de l'individu ;
- **classe** : groupe ou classe d'appartenance.

2.1 Création du jeu de données

2.1.1 Avec Pandas

```
import pandas as pd
data = pd.DataFrame({
    "age": [18, 20, 19, 21, 22, 20, 19, 23],
    "note": [12, 15, 9, 14, 17, 10, 8, 16],
    "sexe": ["F", "M", "F", "M", "F", "M", "F", "M"],
    "classe": ["A", "A", "B", "B", "A", "B", "A", "B"]
})
```

2.1.2 Utilisation de NumPy

Il est possible de définir chaque variable sous forme de vecteur, puis de les regrouper en un tableau de données.

```
import numpy as np
age = np.array([18, 20, 19, 21, 22, 20, 19, 23])
note = np.array([12, 15, 9, 14, 17, 10, 8, 16])
data = np.column_stack((age, note))
```

Interprétation : NumPy permet de regrouper des variables numériques sous forme de tableau.

2.1.3 Limites de NumPy et intérêt de Pandas

La bibliothèque NumPy est particulièrement efficace pour manipuler des tableaux numériques. Elle permet d'effectuer des calculs rapides et vectorisés sur des données quantitatives.

• Limite de NumPy

Lorsque l'on introduit des variables qualitatives, comme une variable **classe**, NumPy convertit automatiquement toutes les données en chaîne de caractères :

```
classe = np.array(["A", "A", "B", "B", "A", "B", "A", "B"])
data = np.column_stack((age, note, classe))
```

Remarque 2.1 Dans ce cas, toutes les valeurs (y compris les variables numériques) sont converties en texte, ce qui peut poser des problèmes pour les traitements statistiques.

• Solution avec Pandas

La bibliothèque Pandas permet de gérer des données hétérogènes (numériques et qualitatives) de manière structurée :

```
import pandas as pd
data = pd.DataFrame({
    "age": age,
    "note": note,
    "classe": ["A", "A", "B", "B", "A", "B", "A", "B"]
})
```

Avantages :

- Conservation du type de chaque variable ;
- Association de noms aux colonnes ;
- Facilité des opérations statistiques et des filtrages ;
- Meilleure lisibilité du tableau.

• Conclusion

NumPy est adapté aux calculs numériques intensifs, tandis que Pandas est plus approprié pour la manipulation et l'analyse de données réelles, souvent hétérogènes.

2.2 Aperçu du jeu de données

Le jeu de données peut être affiché dans son intégralité à l'aide de la commande suivante :

```
print(data) # Affiche l'ensemble du jeu de données
```

Cette commande permet d'afficher l'ensemble du tableau. Toutefois, lorsque le jeu de données est volumineux, il est préférable d'utiliser des fonctions d'exploration adaptées, qui seront présentées dans la section suivante.

Ce jeu de données servira de support pour illustrer les différentes opérations de manipulation, d'analyse et de visualisation présentées dans ce chapitre.

2.3 Lecture de fichiers de données

Dans la pratique, les données ne sont pas créées manuellement mais sont généralement stockées dans des fichiers externes (CSV, TXT, Excel, etc.).

La bibliothèque Pandas permet de lire facilement ces fichiers.

2.3.1 Lecture d'un fichier CSV

```
data = pd.read_csv("donnees.csv")
```

Remarque 2.2 Le format CSV (Comma-Separated Values) est l'un des formats les plus utilisés pour stocker des données tabulaires.

2.3.2 Lecture d'un fichier texte

```
data = pd.read_csv("donnees.txt", sep="\t")
```

Interprétation : L'argument `sep` permet de spécifier le séparateur utilisé dans le fichier (tabulation, point-virgule, etc.).

2.3.3 Cas d'un séparateur point-virgule

```
data = pd.read_csv("donnees.csv", sep=";")
```

Remarque 2.3 Dans certains pays (notamment en Europe), les fichiers CSV utilisent le point-virgule comme séparateur.

2.3.4 Lecture d'un fichier Excel

Les données peuvent également être stockées dans des fichiers Excel (.xlsx). La bibliothèque Pandas permet de lire ces fichiers facilement.

```
import pandas as pd
data = pd.read_excel("donnees.xlsx")
```

Interprétation : Cette commande permet de lire le contenu du fichier Excel et de le convertir en DataFrame.

Remarque 2.4 Par défaut, Pandas lit la première feuille du fichier Excel.

Remarque 2.5 Il est possible de spécifier une feuille particulière à l'aide de l'argument `sheet_name` :

```
data = pd.read_excel("donnees.xlsx", sheet_name="Feuille1")
```

Remarque 2.6 Si le fichier contient plusieurs feuilles, on peut les charger simultanément :

```
data = pd.read_excel("donnees.xlsx", sheet_name=None)
```

Dans ce cas, `data` est un dictionnaire contenant un DataFrame par feuille.

Remarque 2.7 Comme pour les fichiers CSV, la première ligne est interprétée comme les noms des variables.

Remarque 2.8 En cas d'erreur de lecture, il peut être nécessaire d'installer le module `openpyxl` :

```
pip install openpyxl
```

Remarque 2.9 Une vérification systématique du jeu de données après importation est indispensable. Les méthodes correspondantes seront détaillées dans la section consacrée à l'exploration des données.

2.3.5 Bonnes pratiques

- Vérifier le séparateur du fichier ;
- Contrôler les types de variables ;
- Identifier les valeurs manquantes ;
- Nettoyer les données après importation.

3 Exploration initiale des données

Avant toute analyse statistique, il est indispensable de comprendre la structure du jeu de données. Cette étape préliminaire permet d'identifier les caractéristiques essentielles du tableau, de détecter d'éventuelles anomalies et d'orienter les traitements ultérieurs.

L'exploration initiale constitue ainsi une phase fondamentale dans toute démarche d'analyse de données, car elle conditionne la qualité et la pertinence des résultats obtenus.

3.1 Visualisation des premières observations

```
data.head()    # Affiche les 5 premières lignes du tableau
```

Interpretation : Cette commande fournit un aperçu rapide du jeu de données. Elle permet de vérifier la cohérence des variables, d'identifier leur nature et de détecter d'éventuelles erreurs de saisie.

3.2 Visualisation des dernières observations

```
data.tail()    # Affiche les 5 dernières lignes du tableau
```

Interprétation : Cette commande permet d'examiner les dernières observations du tableau. Elle est utile pour vérifier la fin du jeu de données et détecter d'éventuelles anomalies ou valeurs inattendues.

Remarque 3.1 *Il est possible de spécifier le nombre de lignes à afficher en argument :*

```
data.head(4)    # Affiche les 4 premières lignes  
data.tail(3)    # Affiche les 3 dernières lignes
```

Par défaut, ces fonctions affichent 5 lignes.

3.3 Dimension du tableau

```
data.shape     # Donne (nombre de lignes, nombre de colonnes)
```

Interprétation : Cette information permet de connaître :

- le nombre d'observations (lignes) ;
- le nombre de variables (colonnes).

Elle permet également d'apprécier la taille du jeu de données et d'évaluer sa représentativité.

3.4 Identification des variables

```
data.columns    # Liste les noms des variables
```

Interprétation : Cette commande permet d'identifier les variables disponibles et de vérifier leur dénomination. Une bonne compréhension des variables est indispensable pour éviter des erreurs d'interprétation.

Remarque 3.2 *Les noms de variables peuvent contenir des espaces invisibles, ce qui peut provoquer des erreurs lors de leur utilisation.*

Il est recommandé de nettoyer les noms de colonnes avec :

```
data.columns = data.columns.str.strip()
```

3.5 Informations générales sur les données

```
data.info()    # Donne les types et les valeurs manquantes
```

Interprétation : Cette commande fournit des informations essentielles :

- le type de chaque variable (numérique, texte, etc.) ;
- le nombre de valeurs non manquantes ;
- la présence de valeurs manquantes.

Elle permet de détecter rapidement des problèmes de typage ou de qualité des données.

3.6 Bonnes pratiques

- Toujours explorer les données avant toute transformation ;
- Vérifier la cohérence et la signification des variables ;
- Identifier les valeurs manquantes et les anomalies ;
- S'assurer de la qualité des données avant toute analyse statistique.

3.7 Synthèse : première exploration des données

Après importation d'un jeu de données, les commandes suivantes permettent d'obtenir rapidement une vision globale :

```
data.head()      # Aperçu des premières lignes
data.shape       # Dimensions du tableau
data.columns     # Noms des variables
data.info()      # Types et valeurs manquantes
```

Ces commandes constituent un premier réflexe indispensable pour comprendre la structure d'un jeu de données.

3.8 Conclusion

L'exploration initiale permet de comprendre la structure du jeu de données et constitue une étape préparatoire essentielle avant le nettoyage et la transformation.

4 Nettoyage des données

Le nettoyage des données constitue une étape essentielle de toute analyse statistique. Les données réelles sont souvent imparfaites : elles peuvent contenir des valeurs manquantes, des erreurs de saisie ou des incohérences.

Une mauvaise gestion de ces problèmes peut conduire à des résultats biaisés.

4.1 Identification des problèmes

Avant toute correction, il est nécessaire d'explorer les données.

```
data.info()      # Informations générales sur le tableau
data.isnull().sum() # Nombre de valeurs manquantes par variable
```

Ces commandes permettent de détecter les variables contenant des valeurs manquantes.

Exemple de jeu de données avec valeurs manquantes

Les jeux de données réels contiennent souvent des valeurs manquantes. Celles-ci sont représentées dans Pandas par NaN (Not a Number).

```
import pandas as pd
import numpy as np

data1 = pd.DataFrame({
    "age": [18, 20, np.nan, 21, 22, 20],
    "note": [12, 15, 9, np.nan, 17, 10],
    "sexe": ["F", "M", "F", "M", np.nan, "M"],
    "classe": ["A", "A", "B", "B", "A", np.nan]
})
```

Interprétation : Les valeurs NaN indiquent l'absence d'information pour certaines observations.

4.2 Gestion des valeurs manquantes

Les valeurs manquantes peuvent être traitées de plusieurs manières selon le contexte.

4.2.1 Suppression des observations

```
data = data.dropna() # Supprime les lignes contenant des valeurs  
manquantes
```

Cette méthode est adaptée lorsque le nombre de valeurs manquantes est faible.

Limite : Elle peut entraîner une perte d'information si les données manquantes sont nombreuses.

4.2.2 Remplacement des valeurs manquantes

```
data = data.fillna(0) # Remplace les valeurs manquantes par 0
```

Cette méthode permet de conserver toutes les observations.

Attention : Le choix de la valeur de remplacement doit être justifié.

4.2.3 Remplacement par la moyenne

```
data["note"] = data["note"].fillna(data["note"].mean())  
# Remplace les valeurs manquantes par la moyenne
```

Interprétation : On suppose que la valeur manquante est proche du comportement moyen.

4.3 Correction des types de variables

Les variables peuvent être mal interprétées (texte au lieu de numérique, etc.).

```
data.dtypes # Type des variables
```

```
data["age"] = data["age"].astype(int) # Conversion en entier
```

Une mauvaise typologie peut empêcher les calculs statistiques.

4.4 Détection des valeurs aberrantes

Les valeurs aberrantes (outliers) peuvent fortement influencer les résultats.

```
data.describe() # Détection des valeurs extrêmes
```

Exemple : Une valeur très élevée ou très faible peut être suspecte.

4.5 Traitement des valeurs aberrantes

Plusieurs approches sont possibles :

- suppression des valeurs extrêmes
- remplacement par une valeur seuil
- analyse spécifique de ces observations

4.6 Suppression des doublons

Les données peuvent contenir des lignes dupliquées.

```
data = data.drop_duplicates() # Suppression des doublons
```

4.7 Normalisation des données

Dans certains cas, il est utile de transformer les variables.

```
data["note_norm"] = (data["note"] - data["note"].mean()) / data["note"].std() # Normalisation (centrage-réduction)
```

Cette transformation est utile pour les analyses multivariées.

4.8 Bonnes pratiques

- Toujours analyser les données avant de les modifier
- Justifier les choix de traitement
- Eviter les corrections arbitraires
- Documenter les transformations effectuées

4.9 Exemple complet

```
data.info()
data.isnull().sum()

data = data.dropna()

data["age"] = data["age"].astype(int)

data = data.drop_duplicates()

data.head()
```

4.10 Conclusion

Le nettoyage des données est une étape critique de l'analyse statistique. Il conditionne la qualité des résultats obtenus et doit être réalisé avec rigueur.

5 Manipulation des variables

La manipulation des variables constitue une étape essentielle dans l'analyse des données. Elle permet de sélectionner, transformer, créer ou supprimer des variables afin d'adapter le jeu de données à l'objectif de l'étude statistique.

Cette étape joue un rôle central dans la préparation des données et conditionne la pertinence des analyses ultérieures.

5.1 Sélection de variables

La sélection consiste à extraire une ou plusieurs variables (colonnes) du tableau.

5.1.1 Sélection d'une seule variable

```
data["note"] # Sélection d'une seule variable
```

Interprétation : Cette commande retourne une série correspondant à la variable `note`.

5.1.2 Sélection de plusieurs variables

```
data[["age", "note"]] # Sélection de plusieurs variables
```

Interprétation : On obtient un sous-tableau (DataFrame) contenant uniquement les variables sélectionnées.

5.1.3 Remarque importante

Une seule paire de crochets `[]` retourne une série, tandis que deux paires `[][]` retournent un DataFrame. Cette distinction est fondamentale pour la suite des traitements.

5.2 Création de nouvelles variables

Il est souvent nécessaire de créer de nouvelles variables à partir des variables existantes afin d'enrichir l'analyse statistique.

```
data["admis"] = data["note"] >= 10  
# Crée une variable booléenne : True si note >= 10
```

Interprétation : Cette transformation permet de passer d'une variable quantitative en une variable qualitative binaire, facilitant ainsi certaines analyses (classification, filtrage, etc.).

5.2.1 Exemples

```
data["note_double"] = data["note"] * 2 # Multiplie la note par 2  
  
data["reussi"] = data["note"] >= 12 # Définit un seuil de réussite plus exigeant  
reussi_temp = data["note"] >= 12 # Variable Python (non ajoutée au DataFrame)
```

Intérêt :

- construction d'indicateurs ;
- simplification de l'analyse ;
- meilleure interprétation des résultats.

Remarque 5.1 Une nouvelle variable est ajoutée au DataFrame uniquement si elle est définie sous la forme `data["nom"] = ...`.

Une affectation simple à une variable (par exemple `admis = ...`) ne modifie pas le jeu de données.

5.3 Suppression de variables

Certaines variables peuvent devenir inutiles ou redondantes.

```
data = data.drop(columns=["age"])
```

Interprétation : Cette commande permet de supprimer la variable `age` du tableau.

Remarque 5.2 La méthode `drop()` ne modifie pas directement le DataFrame par défaut. Elle renvoie un nouveau tableau.

Si le résultat n'est pas affecté :

```
data.drop(columns=["age"]) # Ne modifie pas data
```

le tableau initial reste inchangé.
Pour appliquer la modification :

```
data = data.drop(columns=["age"])
# ou
data.drop(columns=["age"], inplace=True)
```

5.4 Exemple complet

```
sous_table = data[["age", "note"]]
sous_table["admis"] = sous_table["note"] >= 10
sous_table = sous_table.drop(columns=["age"])
sous_table.head()
```

Interprétation : Cet exemple illustre une chaîne de traitement complète : sélection, transformation et simplification du jeu de données.

5.5 Sélection par indices et sous-échantillonnage

Outre la sélection par nom de variables, il est possible de sélectionner des observations en fonction de leur position dans le tableau.

5.5.1 Sélection par position

```
data.iloc[2:9] # Sélection des individus du 3eme au 9eme (indices 2 à 8)
```

Remarque 5.3 En Python, l'indexation commence à 0. Ainsi, l'indice 2 correspond au troisième individu.

5.5.2 Sélection d'un individu sur trois

```
data.iloc[::3] # Sélection d'un individu sur trois à partir du premier
```

Interprétation : Le pas de 3 permet de sélectionner les lignes 0, 3, 6, etc.

5.5.3 Sélection avec point de départ spécifique

```
data.iloc[2::3] # Sélection d'un individu sur trois à partir du troisième
```

Interprétation : Sélection des lignes 2, 5, 8, etc.

5.5.4 Sélection de lignes et colonnes

```
data.iloc[2:6, 0:2] # Lignes 3 à 6 et colonnes 1 à 2
```

5.6 Sélection par étiquettes

```
data.loc[2:5] # Sélection basée sur les étiquettes d'index
```

5.7 Comparaison entre loc et iloc

La sélection des données dans **Pandas** peut s'effectuer selon deux logiques :

- **iloc** : sélection par position (indices entiers) ;
- **loc** : sélection par étiquettes (index et noms de colonnes).

Exemple. Considérons le DataFrame suivant :

```
data = pd.DataFrame({
    "age": [18, 20, 19],
    "note": [12, 15, 9]
})
```

Si l'on modifie l'index :

```
data.index = ["a", "b", "c"]
```

Alors :

```
data.iloc[1, 1]
```

sélectionne la valeur située en position (1,1), indépendamment des étiquettes.

En revanche,

```
data.loc["b", "note"]
```

sélectionne la valeur correspondant à l'étiquette de ligne "b" et à la colonne "note".

Attention aux tranches. Une différence importante concerne les intervalles :

- `data.iloc[0:2]` : borne droite exclue (convention Python) ;
- `data.loc["a":"b"]` : borne droite incluse.

Conclusion. `iloc` est adapté aux sélections positionnelles. `loc` est préférable lorsque l'index a une signification sémantique (dates, identifiants, catégories).

Remarque 5.4 *La sélection par indices permet de construire des sous-échantillons et d'extraire des portions spécifiques du jeu de données. Elle est essentielle pour les analyses statistiques et les traitements de données.*

5.8 Sélection aléatoire

La sélection aléatoire consiste à extraire un sous-ensemble d'observations de manière aléatoire. Elle est essentielle pour construire des échantillons représentatifs et pour réaliser des analyses statistiques fiables.

5.8.1 Sélection de quelques observations

```
data.sample(n=3) # Sélection aléatoire de 3 individus
```

Interprétation : Cette commande permet de tirer un échantillon de taille fixe de manière aléatoire.

5.8.2 Sélection d'une proportion du jeu de données

```
data.sample(frac=0.5) # Sélection de 50% des observations
```

Interprétation : On sélectionne une fraction du jeu de données.

5.8.3 Reproductibilité des résultats

```
data.sample(n=3, random_state=1)
```

Remarque 5.5 *Le paramètre `random_state` permet de reproduire le même échantillon aléatoire.*

5.8.4 Echantillonnage avec remise

```
data.sample(n=5, replace=True) # Tirage avec remise
```

Interprétation : Une même observation peut être sélectionnée plusieurs fois.

5.8.5 Intérêt statistique

La sélection aléatoire est utilisée pour :

- construire des échantillons représentatifs ;
- réduire la taille des données ;
- préparer des analyses statistiques ou des modèles ;
- éviter les biais de sélection.

Remarque 5.6 *La sélection aléatoire constitue un outil fondamental en statistique. Elle permet de garantir la validité des analyses et de travailler efficacement sur des sous-ensembles de données.*

5.9 Bonnes pratiques

- Donner des noms explicites aux variables créées ;
- Eviter d'écraser des variables importantes ;
- Vérifier systématiquement les résultats intermédiaires ;
- Maintenir une cohérence dans la structure du tableau.

5.10 Conclusion

La manipulation des variables permet d'adapter les données aux besoins de l'analyse. Elle constitue une étape fondamentale dans tout processus de traitement de données et conditionne la qualité des résultats obtenus.

6 Filtrage des données

Le filtrage des données est une opération fondamentale en analyse statistique. Il permet d'extraire une sous-population à partir d'un ensemble de données initial en fonction de conditions logiques.

6.1 Filtrage simple

```
data[data["note"] >= 10] # Sélection des individus ayant une note >= 10
```

Cette commande sélectionne les observations vérifiant la condition imposée.

6.2 Filtrage avec plusieurs conditions

```
data[(data["note"] >= 10) & (data["age"] >= 20)] # Sélection : note >= 10  
ET age >= 20
```

Chaque condition doit être entourée de parenthèses.

6.3 Filtrage avec condition OU

```
data[(data["note"] >= 15) | (data["age"] < 18)]# Sélection : note élevée  
OU age faible
```

6.4 Filtrage sur variable qualitative

```
data[data["sexe"] == "F"]# Sélection des individus de sexe féminin
```

6.5 Filtrage avec négation

```
data[data["note"] != 10]# Sélection des individus dont la note est diffé-  
rente de 10
```

6.6 Stockage du résultat

```
admis = data[data["note"] >= 10]
```

6.7 Bonnes pratiques

- Utiliser des parenthèses pour les conditions multiples
- Vérifier les résultats avec `data.head()`
- Donner des noms explicites aux sous-tableaux

6.8 Exemple

```
admis = data[data["note"] >= 10]  
admis.head()
```

6.9 Conclusion

Le filtrage permet de focaliser l'analyse sur une population spécifique et constitue une étape essentielle en statistique appliquée.

7 Statistiques descriptives

L'analyse descriptive constitue une étape fondamentale dans l'étude des données. Elle permet de résumer l'information contenue dans un jeu de données à l'aide d'indicateurs numériques.

Ces indicateurs permettent de décrire :

- la tendance centrale ;
- la dispersion ;
- la position relative dans la distribution ;
- la forme de la distribution.

7.1 Mesures de tendance centrale

Les mesures de tendance centrale caractérisent la valeur typique d'une variable.

```
data.mean()      # Moyenne
data.median()    # Médiane
data.mode()      # Mode
```

Interpretation :

- La moyenne est sensible aux valeurs extrêmes.
- La médiane est robuste aux valeurs aberrantes.
- Le mode correspond à la valeur la plus fréquente.

7.2 Mesures de position

Les mesures de position permettent de situer une observation dans la distribution.

```
data.quantile(0.25) # Premier quartile
data.quantile(0.5)  # Médiane
data.quantile(0.75) # Troisième quartile
```

Interprétation :

- Les quartiles divisent la distribution en quatre parties égales.
- Les quantiles permettent une analyse plus fine de la distribution.

7.3 Mesures de dispersion

Les mesures de dispersion quantifient la variabilité des données.

```
data.var()      # Variance
data.std()      # Ecart-type
```

Interprétation :

- Une variance élevée traduit une forte dispersion.
- L'écart-type est plus interprétable car exprime dans la même unité.

7.4 Etendue et coefficient de variation

```
data.max() - data.min() # Etendue
data.std() / data.mean() # Coefficient de variation
```

Interprétation :

- L'étendue mesure l'écart entre les valeurs extrêmes.
- Le coefficient de variation permet de comparer la dispersion relative entre variables.

7.5 Mesures de forme

Les mesures de forme caractérisent la distribution des données.

```
data.skew()      # Asymétrie (skewness)
data.kurt()      # Aplatissement (kurtosis)
```

Interprétation :

- Une asymétrie positive indique une distribution étirée vers la droite.
- Une asymétrie négative indique une distribution étirée vers la gauche.
- Une kurtosis élevée indique une distribution très concentrée (pics).
- Une kurtosis faible indique une distribution aplatie.

7.6 Résumé statistique global

```
data.describe()    # Résumé statistique complet
```

Cette commande fournit :

- effectif (count)
- moyenne (mean)
- écart-type (std)
- minimum et maximum
- quartiles (25%, 50%, 75%)

7.7 Analyse et interprétation globale

L'analyse conjointe de ces indicateurs permet :

- d'identifier la structure des données ;
- de détecter des valeurs aberrantes ;
- d'évaluer la symétrie des distributions ;
- de comparer plusieurs variables ;
- de préparer les analyses statistiques ultérieures.

7.8 Résumé des principales commandes

```
# Statistiques descriptives
data.mean()          # Moyenne
data.median()        # Médiane
data.mode()          # Mode

# Dispersion
data.std()           # Écart-type
data.var()           # Variance
data.min()           # Minimum
data.max()           # Maximum

# Quantiles
data.quantile([0.25, 0.5, 0.75])  # Quartiles

# Forme de la distribution
data.skew()          # Asymétrie
data.kurt()          # Kurtosis

# Résumé global
data.describe()

# Valeurs manquantes
data.isnull().sum()
```

7.9 Conclusion

Les statistiques descriptives permettent d'obtenir une vision synthétique des données. Elles constituent une base indispensable pour toute analyse statistique ou économique approfondie.

8 Analyse bivariable

L'analyse bivariable a pour objectif d'étudier la relation entre deux variables. Elle permet de mettre en évidence d'éventuelles dépendances et de mesurer l'intensité des liaisons entre variables.

Deux outils fondamentaux sont utilisés : la covariance et le coefficient de corrélation.

8.1 Covariance

La covariance mesure le sens de variation conjointe de deux variables.

```
data.cov() # Matrice de covariance
```

Définition : Soient X et Y deux variables aléatoires. La covariance est définie par :

$$\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}(X))(Y - \mathbb{E}(Y))]$$

Interprétation :

- Si la covariance est positive, les variables évoluent dans le même sens.
- Si elle est négative, elles évoluent en sens contraire.
- Si elle est proche de zéro, il n'y a pas de liaison linéaire apparente.

Limite : La covariance dépend de l'échelle des variables et ne permet pas une interprétation directe de l'intensité de la relation.

8.2 Coefficient de corrélation

Le coefficient de corrélation permet de mesurer l'intensité de la liaison linéaire entre deux variables.

```
data.corr() # Matrice de corrélation
```

Définition : Le coefficient de corrélation de Pearson est défini par :

$$\rho(X, Y) = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

Propriétés :

- $-1 \leq \rho(X, Y) \leq 1$
- $\rho = 1$: liaison linéaire parfaite croissante
- $\rho = -1$: liaison linéaire parfaite décroissante
- $\rho = 0$: absence de liaison linéaire

Interprétation :

- $|\rho|$ proche de 1 : forte corrélation
- $|\rho|$ proche de 0 : faible corrélation

8.3 Analyse conjointe

La combinaison de la covariance et de la corrélation permet :

- de détecter des relations entre variables ;
- d'identifier des dépendances linéaires ;
- de préparer des analyses plus avancées (régression, modélisation).

8.4 Exemple d'utilisation

```
data.cov()
data.corr()
```

Ces commandes permettent d'obtenir une vision globale des relations entre les variables du jeu de données.

8.5 Conclusion

L'analyse bivariée constitue une étape essentielle dans l'étude des relations entre variables. Elle permet de mieux comprendre la structure des données et de guider les analyses statistiques ultérieures.

9 Visualisation des données

La visualisation des données constitue une étape essentielle de l'analyse statistique. Elle permet de représenter graphiquement les données afin de mieux comprendre leur structure, d'identifier des tendances et de détecter d'éventuelles anomalies.

Le choix du type de représentation dépend de la nature des variables considérées.

9.1 Types de variables et représentations adaptées

- Variables qualitatives : diagramme en barres, diagramme circulaire
- Variables quantitatives discrètes : diagramme en barres, polygone des effectifs
- Variables quantitatives continues : histogramme, courbe cumulative, boîte à moustaches
- Relations entre deux variables : nuage de points

9.2 Histogramme (variable quantitative continue)

```
import matplotlib.pyplot as plt

plt.hist(data["note"])
plt.show()
```

Interprétation : L'histogramme permet de visualiser la distribution des données et d'identifier leur forme (symétrie, asymétrie, dispersion).

9.3 Diagramme en barres

```
data["note"].value_counts().plot(kind="bar")
plt.show()
```

Interprétation : Le diagramme en barres est adapté aux variables discrètes ou qualitatives. Il représente les effectifs ou fréquences de chaque modalité.

9.4 Polygone des effectifs

```
data["note"].value_counts().sort_index().plot()
plt.show()
```

Interprétation : Le polygone des effectifs relie les fréquences et permet de visualiser l'évolution de la distribution.

9.5 Courbe cumulative

9.5.1 Cas discret

```
data["note"].value_counts().sort_index().cumsum().plot()
plt.show()
```

9.5.2 Cas continu

```
data["note"].sort_values().reset_index(drop=True).plot()  
plt.show()
```

Interprétation : La courbe cumulative permet de lire les quantiles et d'analyser la répartition des données.

9.6 Boite a moustaches (boxplot)

```
plt.boxplot(data["note"])  
plt.show()
```

Interprétation : La boîte à moustaches permet de visualiser :

- la médiane;
- les quartiles;
- les valeurs aberrantes.

9.7 Nuage de points

```
plt.scatter(data["age"], data["note"])  
plt.show()
```

Interprétation : Le nuage de points permet d'étudier la relation entre deux variables quantitatives et de détecter une éventuelle corrélation.

9.8 Diagramme circulaire

```
data["sexe"].value_counts().plot(kind="pie")  
plt.show()
```

Interprétation : Le diagramme circulaire représente la répartition des proportions d'une variable qualitative.

9.9 Diagramme en barres comparées

```
data.groupby("sexe")["note"].mean().plot(kind="bar")  
plt.show()
```

Interprétation : Permet de comparer une statistique entre plusieurs groupes.

9.10 Conclusion

La visualisation constitue un outil indispensable pour comprendre les données. Elle permet de compléter l'analyse numérique et d'obtenir une interprétation plus intuitive des résultats.

10 Regroupement des données

Le regroupement des données constitue une opération essentielle en analyse statistique. Il permet de partitionner un jeu de données en sous-groupes homogènes selon les modalités d'une ou plusieurs variables, puis de calculer des statistiques au sein de chaque groupe.

Cette approche est fondamentale pour analyser des différences entre catégories et mettre en évidence des structures dans les données.

10.1 Principe général

Le regroupement repose sur la méthode `groupby()` de la bibliothèque Pandas.

```
data.groupby("sexe")["note"].mean() # Moyenne des notes par sexe
```

Interprétation : Cette commande calcule la moyenne de la variable `note` pour chaque modalité de la variable `sexe`. Elle permet ainsi de comparer les performances moyennes entre groupes.

10.2 Regroupement avec plusieurs statistiques

Il est possible de calculer plusieurs indicateurs simultanément.

```
data.groupby("sexe")["note"].agg(["mean", "std", "min", "max"])
```

Interprétation : On obtient, pour chaque groupe, un résumé statistique incluant :

- la moyenne ;
- l'écart-type ;
- le minimum ;
- le maximum.

10.3 Regroupement selon plusieurs variables

```
data.groupby(["sexe", "classe"])["note"].mean()
```

Interprétation : Cette commande permet d'affiner l'analyse en croisant plusieurs critères de regroupement.

10.4 Application à une variable qualitative

```
data.groupby("sexe").size()  
# Effectif de chaque groupe
```

Interprétation : On obtient le nombre d'observations dans chaque catégorie.

10.5 Transformation des résultats

```
data.groupby("sexe")["note"].mean().reset_index()
```

Intérêt : Permet de convertir le résultat en tableau exploitable pour d'autres traitements ou visualisations.

10.6 Bonnes pratiques

- Choisir des variables de regroupement pertinentes ;
- Interpréter les résultats dans leur contexte ;
- Comparer les groupes de manière rigoureuse ;
- Compléter l'analyse par des représentations graphiques.

10.7 Conclusion

Le regroupement des données permet d'analyser des différences entre groupes et de mettre en évidence des structures internes au jeu de données. Il constitue un outil fondamental en statistique descriptive et en analyse exploratoire.

11 Calculs avancés

Les calculs avancés permettent de transformer les données brutes en indicateurs plus élaborés, facilitant ainsi leur interprétation et leur exploitation dans un cadre statistique ou économique.

Ils reposent sur l'utilisation de fonctions intégrées ou définies par l'utilisateur.

11.1 Taux de croissance

Le taux de croissance constitue un exemple classique de transformation d'une variable temporelle en un indicateur synthétique, largement utilisé en analyse économique.

```
data["croissance"] = data["PIB"].pct_change() * 100
# Calcule le taux de croissance en pourcentage
```

Définition : Soit (X_t) une série temporelle. Le taux de croissance est défini par :

$$g_t = \frac{X_t - X_{t-1}}{X_{t-1}} \times 100$$

Interprétation :

- $g_t > 0$: croissance positive ;
- $g_t < 0$: diminution de la variable ;
- $g_t = 0$: stabilité.

Remarque : La première observation ne possède pas de taux de croissance (valeur manquante).

11.2 Fonctions personnalisées

Il est souvent nécessaire de définir des fonctions adaptées à un besoin spécifique.

```
data["carre"] = data["note"].apply(lambda x: x**2)
# Applique une fonction a chaque valeur
```

Interprétation : La méthode `apply()` permet d'appliquer une transformation à chaque élément d'une variable.

11.2.1 Définition explicite d'une fonction

```
def carre(x):
    return x**2
data["carre"] = data["note"].apply(carre)
```

Intérêt :

- clarté du code ;
- réutilisation de la fonction ;
- adaptation à des transformations complexes.

11.3 Calculs conditionnels

Il est possible de définir des transformations dépendantes de conditions.

```
data["mention"] = data["note"].apply(
    lambda x: "admis" if x >= 10 else "ajourne"
)
```

Interprétation : Cette transformation permet de créer une variable qualitative à partir d'une condition.

11.4 Opérations vectorisées

Les opérations vectorisées sont plus efficaces que les boucles.

```
data["note_double"] = data["note"] * 2
```

Remarque : Les opérations vectorisées sont recommandées pour des raisons de performance.

11.5 Bonnes pratiques

- Privilégier les opérations vectorisées ;
- Utiliser des fonctions claires et réutilisables ;
- Vérifier la cohérence des transformations ;
- Interpréter les résultats dans leur contexte.

12 Conclusion

La manipulation des données à l'aide de la bibliothèque Pandas constitue une étape fondamentale dans toute démarche d'analyse statistique. Elle permet de transformer des données brutes, souvent complexes et imparfaites, en informations structurées et exploitables.

Au cours de ce chapitre, nous avons présenté les principales étapes du traitement des données : exploration initiale, nettoyage, transformation, analyse descriptive, visualisation et regroupement. Chacune de ces étapes contribue à améliorer la qualité des données et à renforcer la pertinence des analyses.

L'utilisation conjointe d'outils informatiques et de concepts statistiques permet ainsi de développer une approche rigoureuse de l'analyse des données, indispensable dans de nombreux domaines tels que l'économie, la finance ou la science des données.

Les compétences acquises dans ce chapitre constituent un socle essentiel pour aborder des méthodes plus avancées, notamment en statistique inférentielle, en économétrie et en apprentissage automatique.

Ainsi, la maîtrise des techniques de manipulation des données ne se limite pas à un aspect technique, mais s'inscrit dans une démarche globale d'analyse, visant à extraire de la connaissance à partir des données.